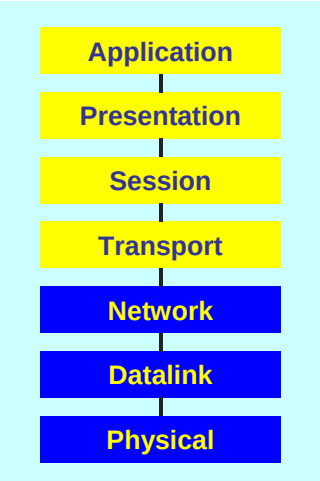
Lecture 3:
Signaling and Clock Recovery

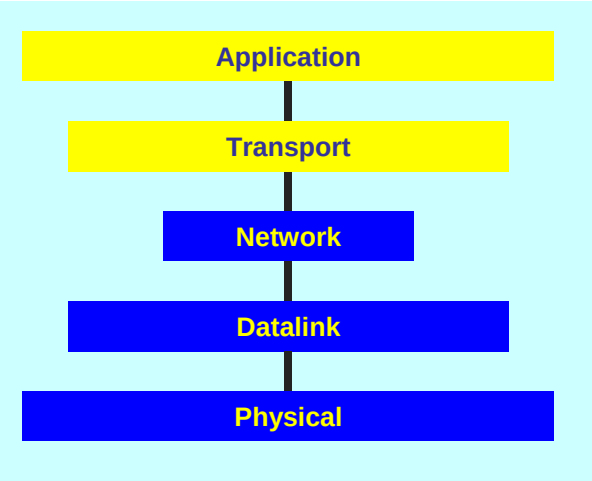
CSE 123: Computer Networks

Stefan Savage

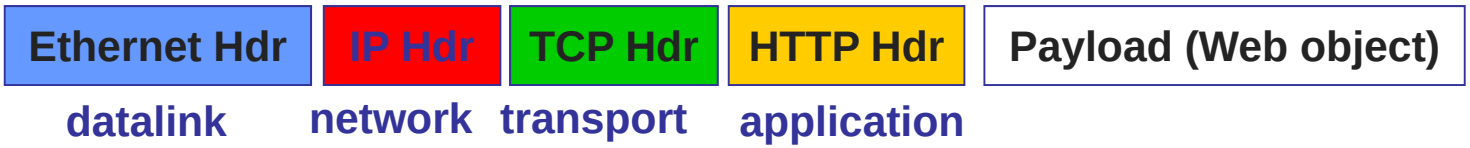
Last time

- Protocols and layering





- Layer encapsulation via packet headers



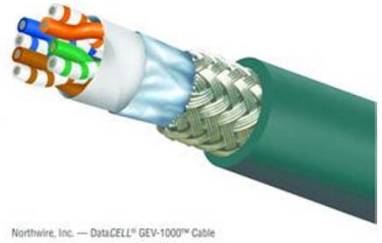
Today

- How to send data from point A to point B over some link?

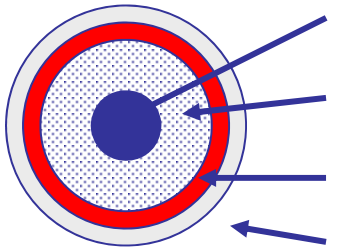
Copper

- Typical examples
  - Category 5 Twisted Pair — 10-1Gbps — 50-100m
  - Coaxial Cable — 10-100Mbps — 200m

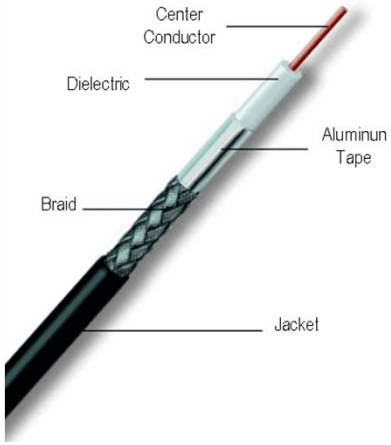
twisted pair



coaxial cable (coax)

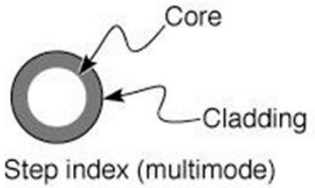


copper core
insulation
braided outer conductor
outer insulation



Fiber Optics

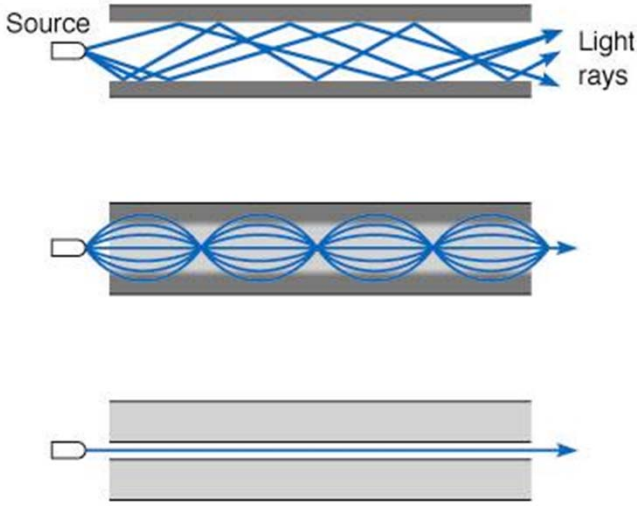
- Typical examples
  - Multimode Fiber 10Gbps 300m
  - Single Mode Fiber 100Gbps 25km



Step index (multimode)


Graded index (multimode)


Single mode

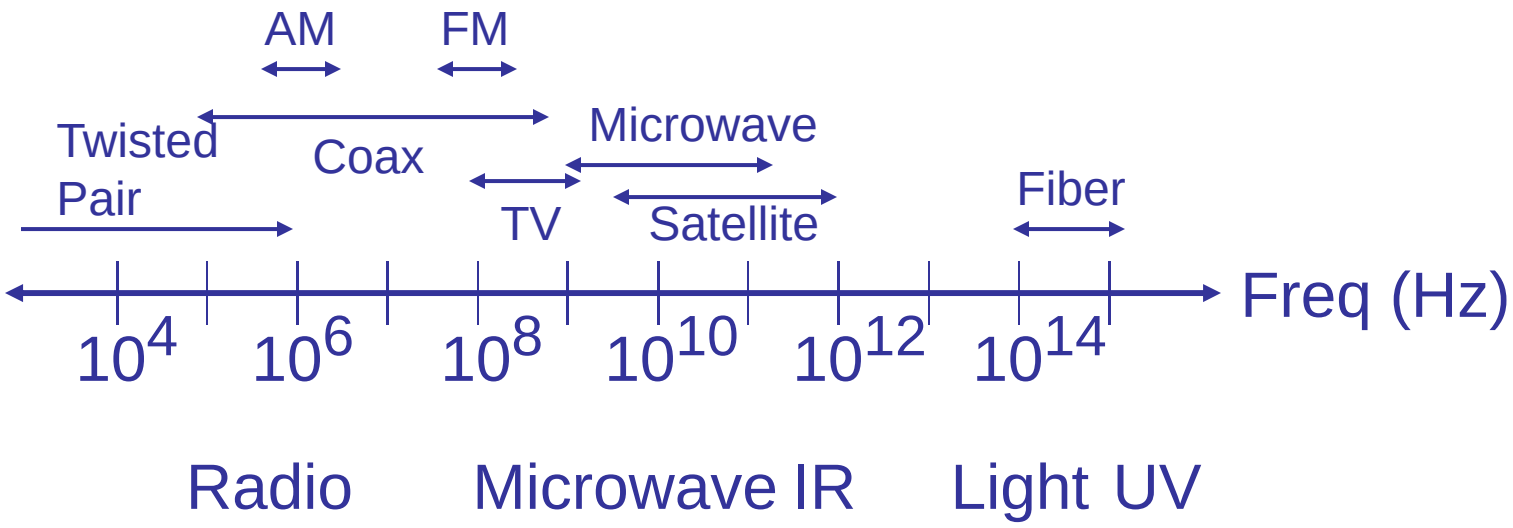


Cheaper to drive (LED vs laser) & terminate

Longer distance (low attenuation)
Higher data rates (low dispersion)

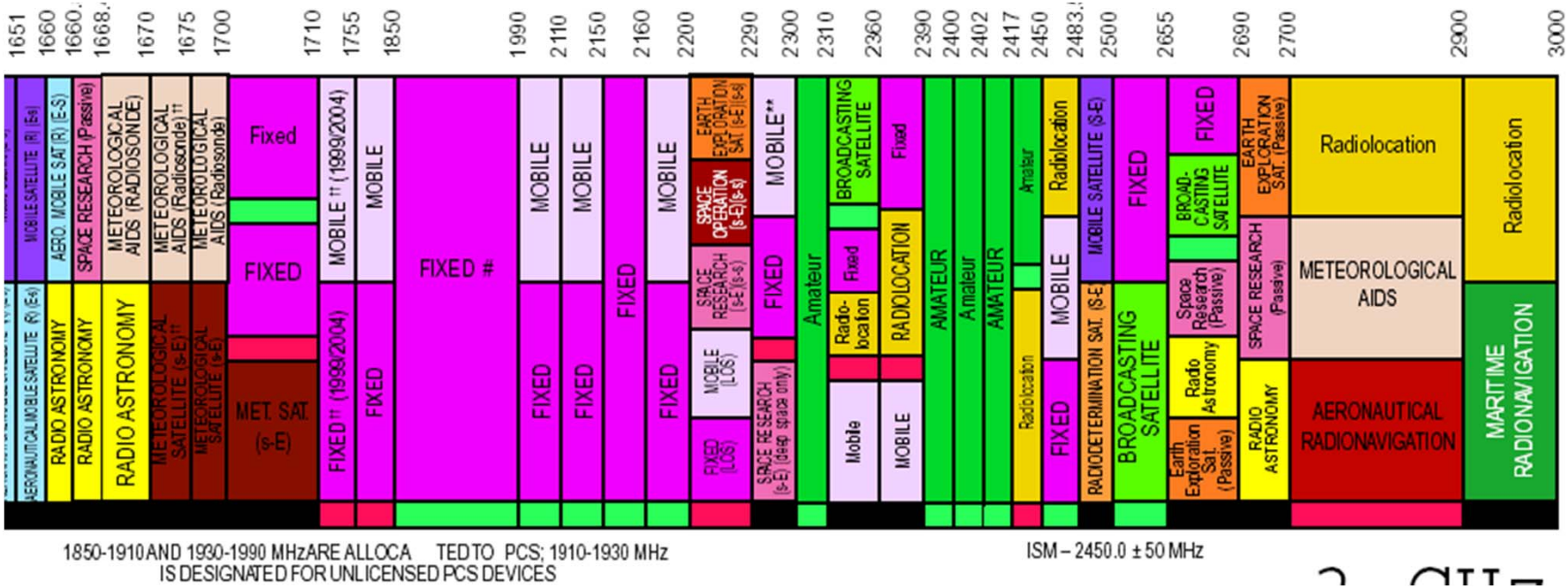
Wireless

- Widely varying channel bandwidths/distances
- Extremely vulnerable to noise and interference



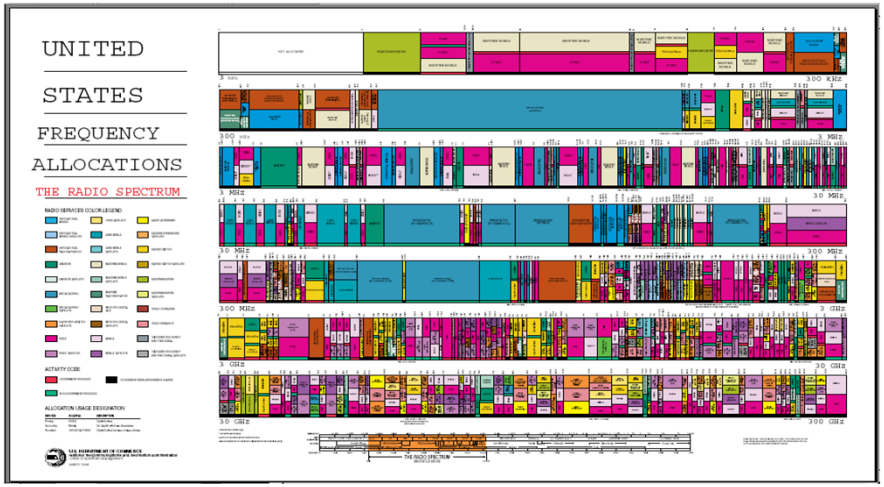
Aside: Wireless spectrum

- Shared medium -> regulated use
- Wireless frequency allocations

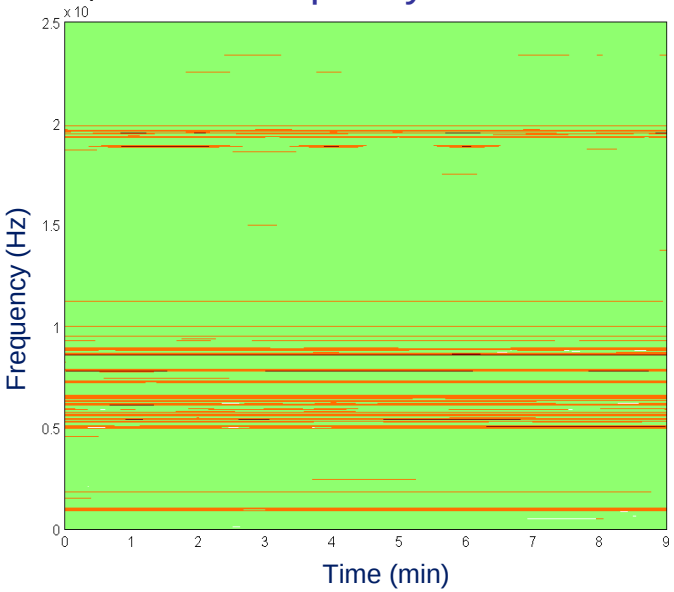


Spectrum Allocation

- Policy approach forces spectrum to be allocated like a fixed spatial resource (e.g. land, disk space, etc)



- Reality is that spectrum is time and power shared
- Measurements show that fixed allocations are poorly utilized



Whitespaces?

Overview for today

- Digital Signaling
  - Why?
  - Shannon's Law and Nyquist Limit
- Encoding schemes
  - Clock recovery
  - Manchester, NRZ, NRZI, etc.
- A lot of this material is not in the book
- Caveat: I am not an EE Professor

Signals:
10,000 foot review of digital vs analog

- Data can be represented continuously as an analog wave

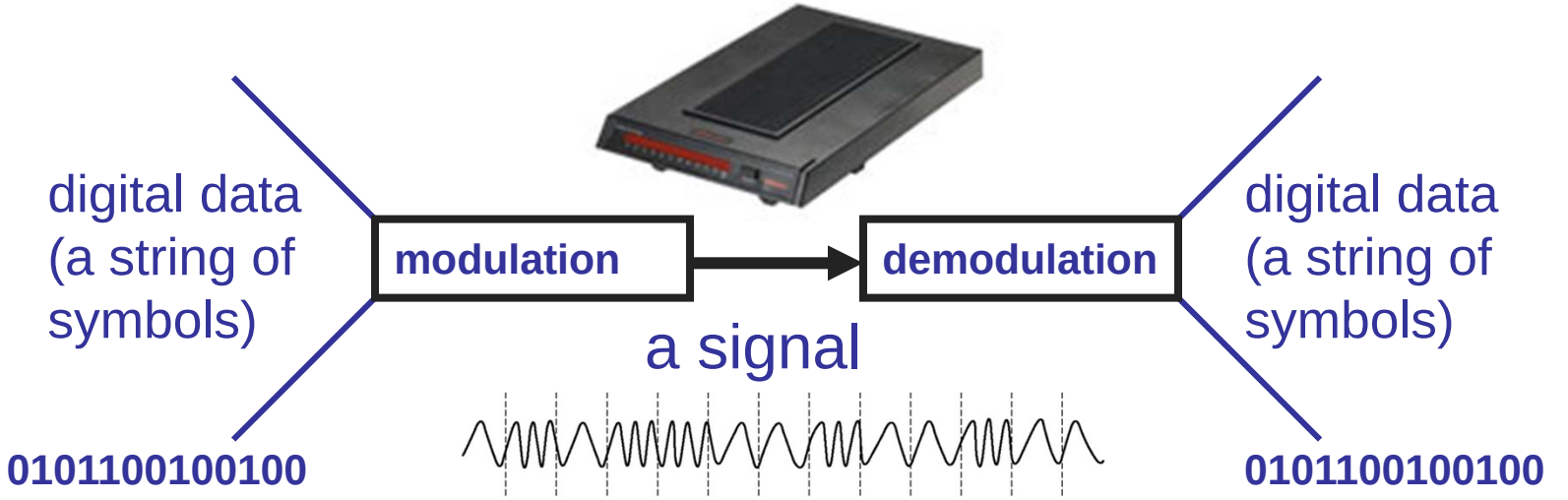
- Or as a series of discrete digital symbols

0 0 1 0 1 1 1 0 0 0 1

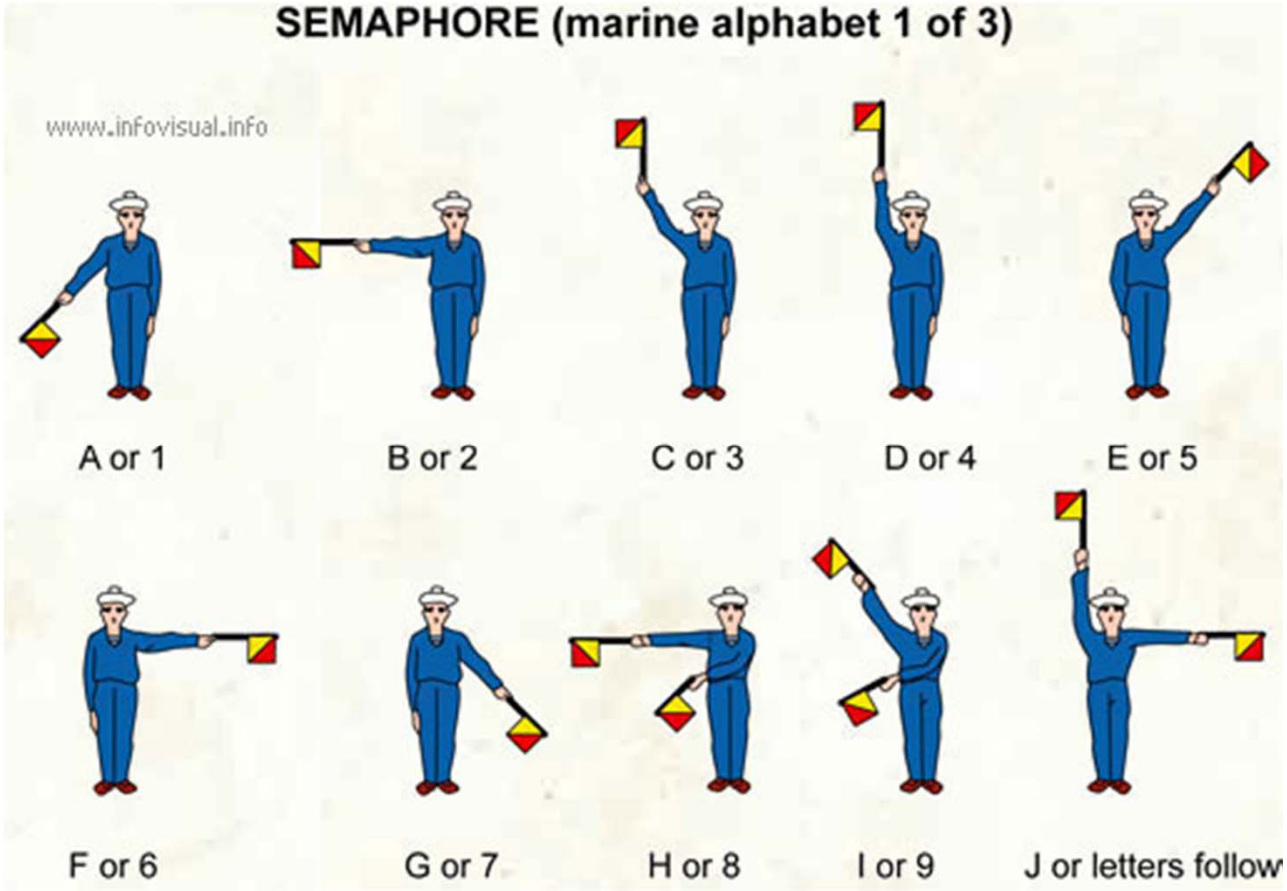
- Why is the digital representation so powerful?
  - Noise resistance: has to be a 0 or 1
  - Storage/reproducibility: analog waves can be converted to digital for storage and then back to analog (think about CDs)

Overall Goal: Send bits

- A three-step process
  - Take an input stream of bits (digital data)
  - Modulate some physical media to send data (analog)
  - Demodulate the signal to retrieve bits (digital again)
  - Anybody heard of a modem (Modulator-demodulator)?



A Simple Signaling System



Signals and Channels

- A signal is some form of energy (light, voltage, etc)
  - Varies with time (on/off, high/low, etc.)
  - Can be continuous or discrete
  - We assume it is periodic with a fixed frequency

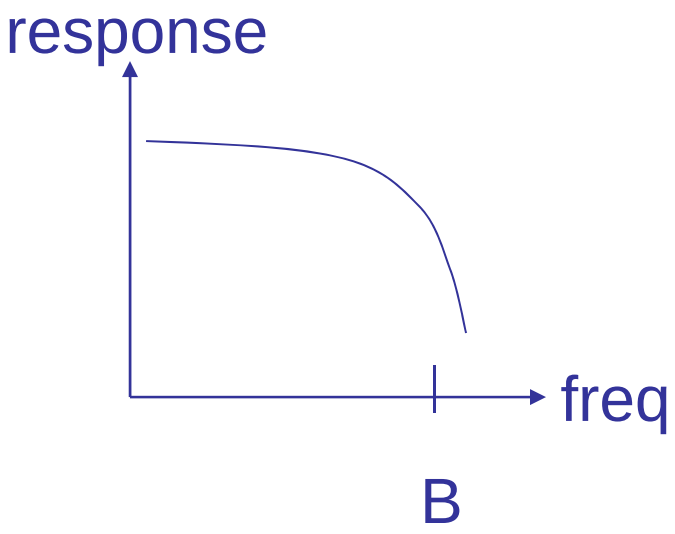
- A channel is a physical medium that conveys energy
  - Any real channel will distort the input signal as it does so
  - How it distorts the signal depends on the signal and the channel…

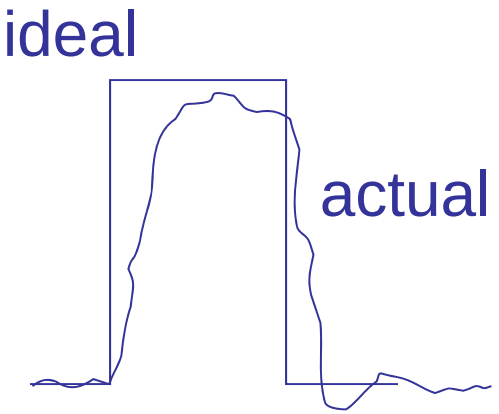
Channel Properties

- Bandwidth-limited
  - Range of frequencies the channel will transmit
  - Means the channel is slow to react to change in signal
- Power attenuates over distance
  - Signal gets softer (harder to “hear”) the further it travels
  - Different frequencies have different response (distortion)
- Background noise or interference
  - May add or subtract from original signal
- Different physical characteristics
  - Point-to-point vs. shared media
  - Very different price points to deploy

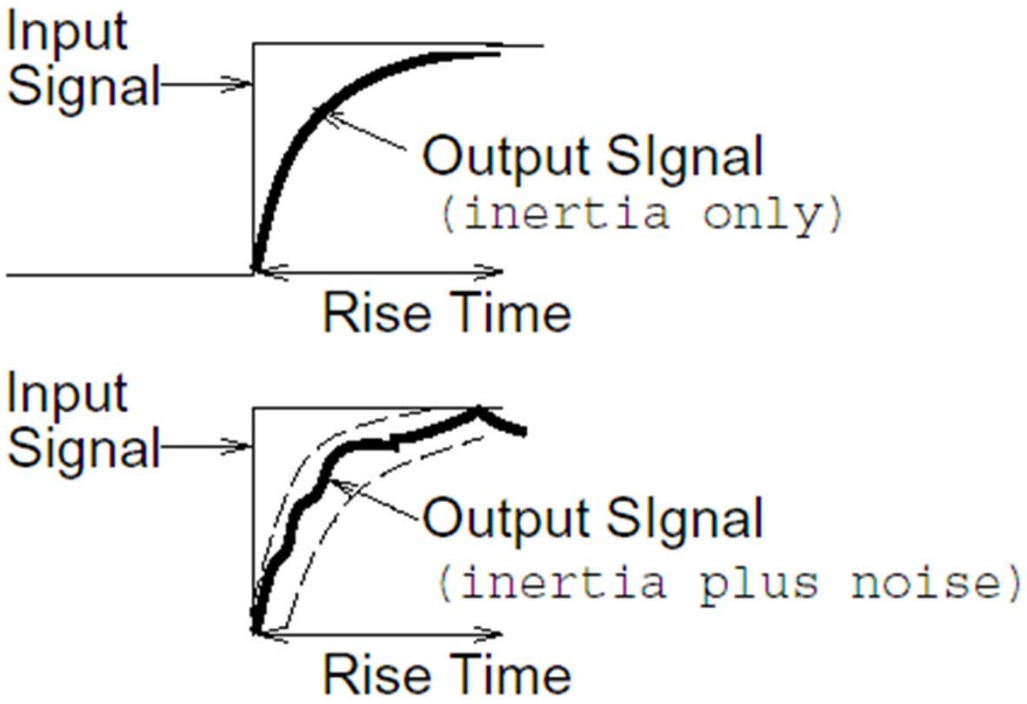
Channel Challenges

- Every channel degrades a signal
  - Distortion impacts how the receiver will interpret signal





Impact of distortion & noise

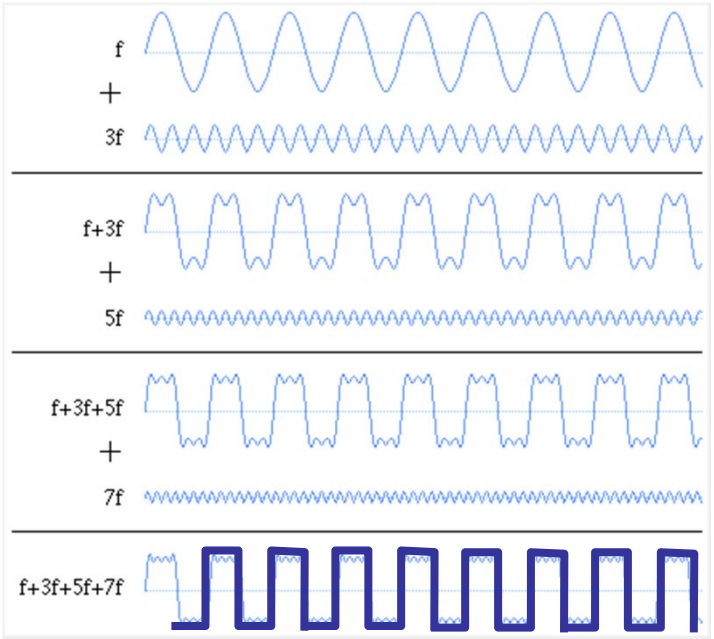


Two Main Tasks

- First we need to transmit a signal
  - Determine how to send the data, and how quickly
- Then we need to receive a (degraded) signal
  - Figure out when someone is sending us bits
  - Determine which bits they are sending
- A lot like a conversation
  - “WhatintheworldamIsaying” – needs punctuation and pacing
  - Helps to know what language I’m speaking

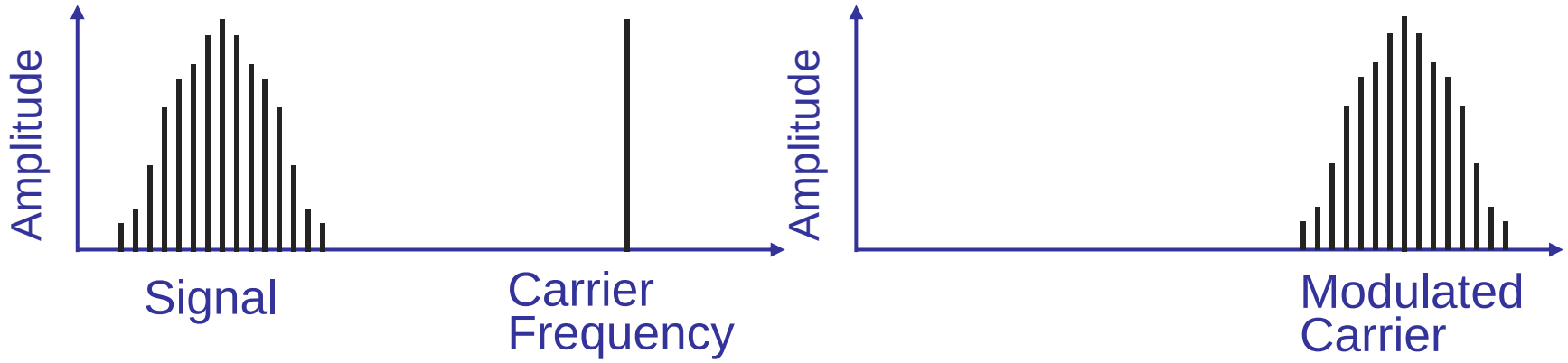
The Magic of Sine Waves

- All periodic signals can be expressed as sine waves
  - Component waves are of different frequencies
- Sine waves are “nice”
  - Phase shifted or scaled by most channels
- “Easy” to analyze
  - Fourier analysis can tell us how signal changes
  - But not in this class…

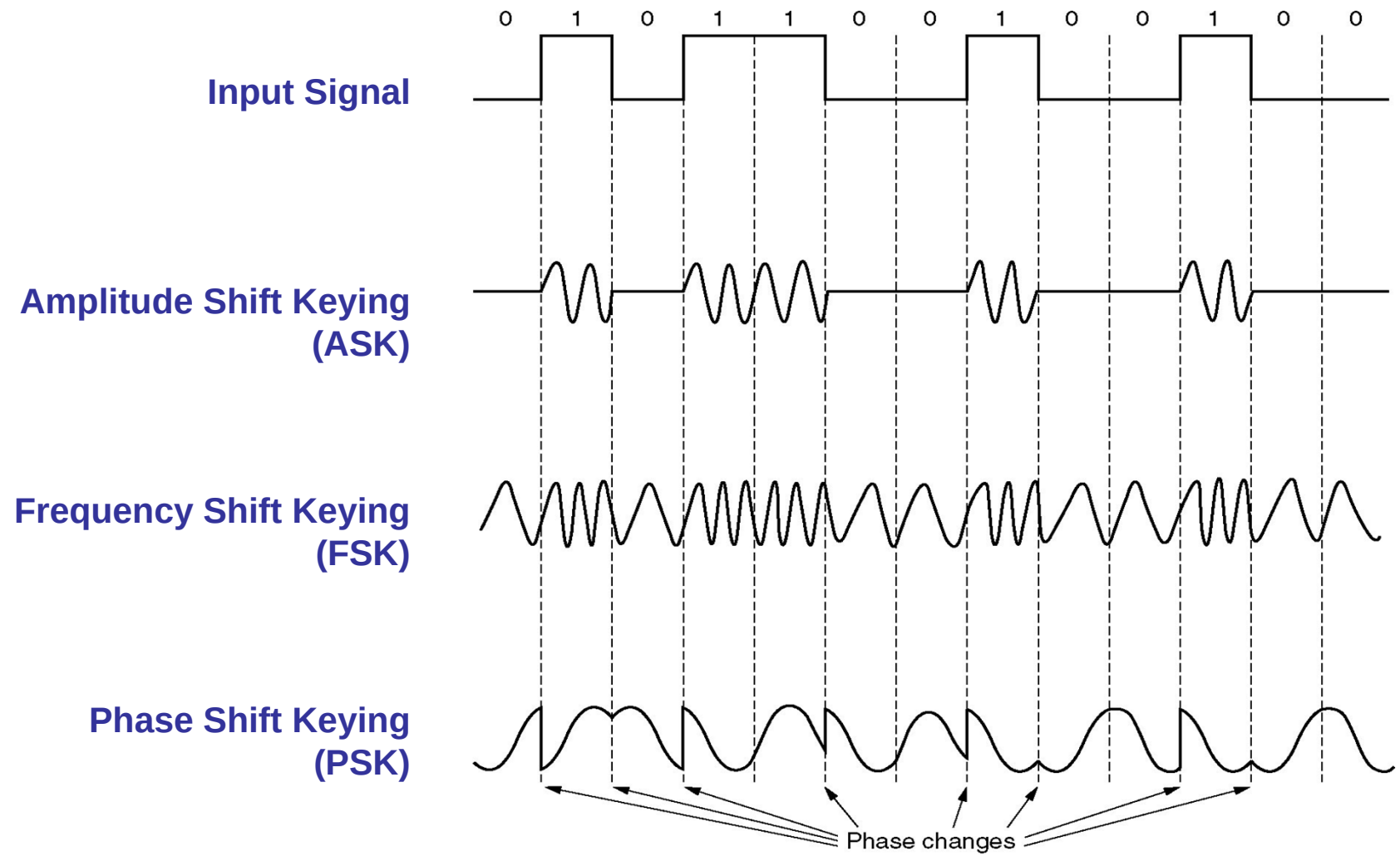


Carrier Signals

- Baseband modulation: send the “bare” signal
  - E.g. +5 Volts for 1, -5 Volts for 0
  - All signals fall in the same frequency range
- Broadband modulation
  - Use the signal to modulate a high frequency signal (carrier).
  - Can be viewed as the product of the two signals



Forms of Digital Modulation

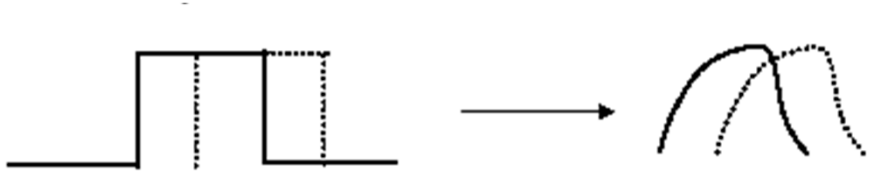


Why Different Schemes?

- Properties of channel and desired application
  - AM vs FM for analog radio
- Efficiency
  - Some modulations can encode many bits for each symbol (subject to Shannon limit)
- Aiding with error detection
  - Dependency between symbols… can tell if a symbol wasn't decoded correctly
- Transmitter/receiver Complexity

Inter-symbol Interference

- Bandlimited channels cannot respond faster than some maximum frequency *f*
  - Channel takes some time to “settle”
- Attempting to signal too fast will mix “symbols”
  - Previous symbol still “settling in”
  - Mix (add/subtract) adjacent symbols
  - Leads to inter-symbol interference (ISI)



- OK, so just how fast can we send symbols?

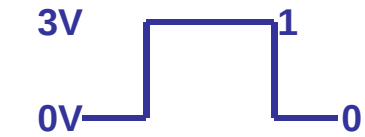
Speed Limit: Nyquist

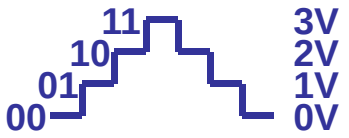
- In a channel bandlimited to frequency *f*, we can send at maximum symbol (baud) rate of *2f* without ISI

- Directly related to Nyquist sampling theorem
  - Can reconstruct signal with maximum frequency f, by sampling at least 2f times per second

Multiple Bits per Symbol

- OK, but why not send multiple bits per symbol?
  - E.g., instead of 3V=1 and 0V=0, multiple levels
    - » 0V=00, 1V=01, 2V=10, 3V=11, etc
  - Four levels gets you two bits, *log L* in general

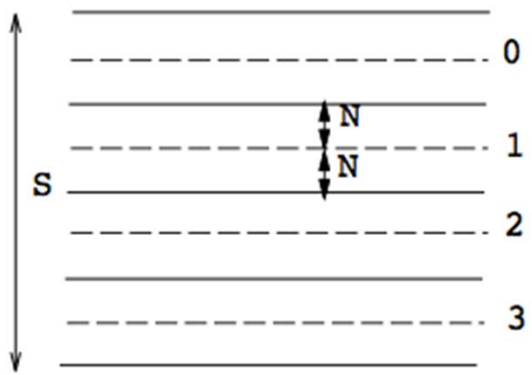




- Could we define an infinite number of levels?

Shannon's law: noise kills

- S= signal power, N= noise
- Channel noise limits bit density
  - Intuitively, need level separation
    - » E.g., what if noise is 0.5V and you read signal of 1.5V.. what symbol is that?
  - Can only get log(*S/2N)* bits per symbol



- Can combine this observation with Nyquist (and some more math)
  - *C = B log (1 + S/N) Shannon/Nyquist Capacity limit*
- *Example: old modems* *(B=3000Hz, S/N=30dB = 1000)*
  - C = 3000 x log(1001) =~ 30kbps

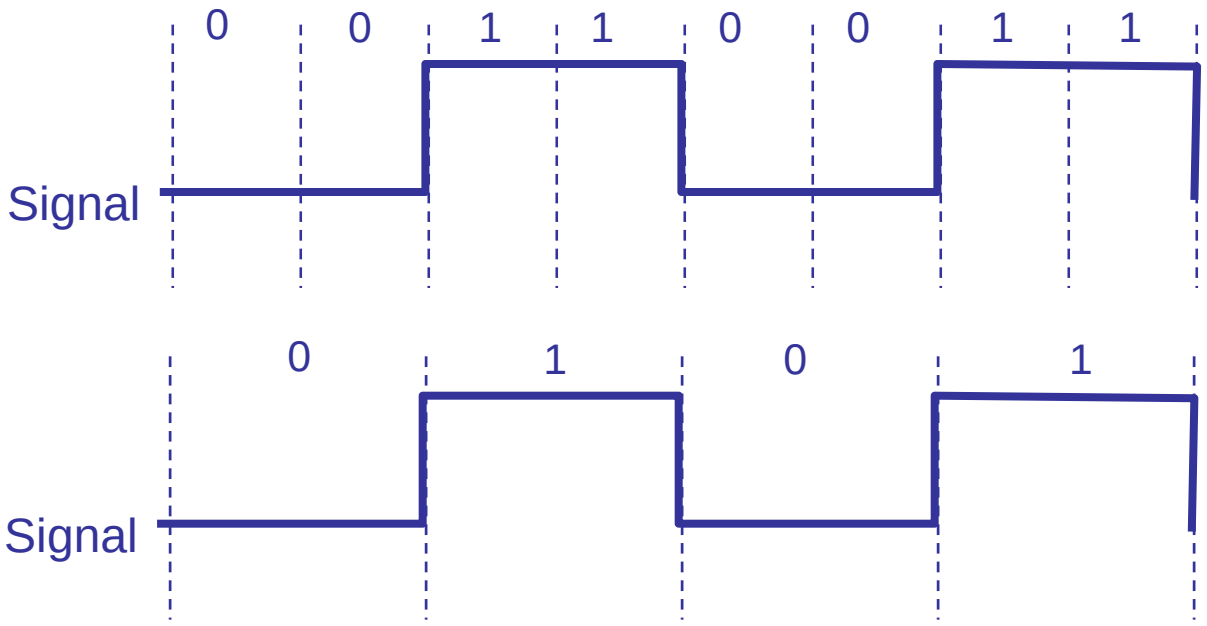
Next problem: Clock recovery

- How does the receiver know when to sample the signal?
  - Sampling rate: How often to sample?
  - Sampling phase:
    - When to start sampling? (getting in phase)
    - How to adjust sampling times (staying in phase)

Sampling at the Receiver

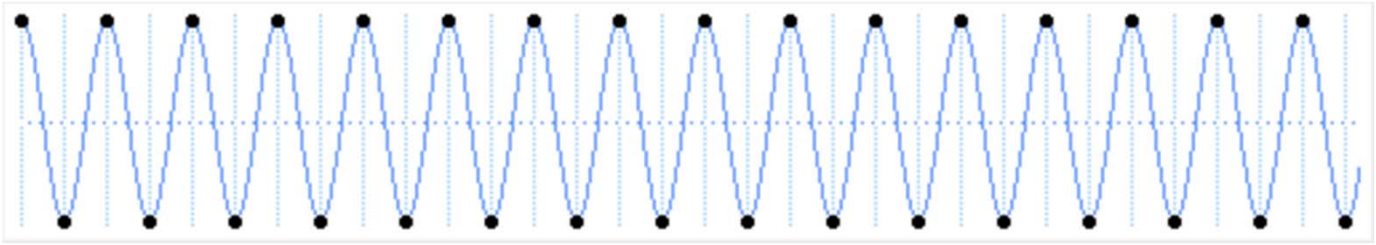
- Need to determine correct sampling frequency
  - Signal could have multiple interpretations

Which of these is correct?

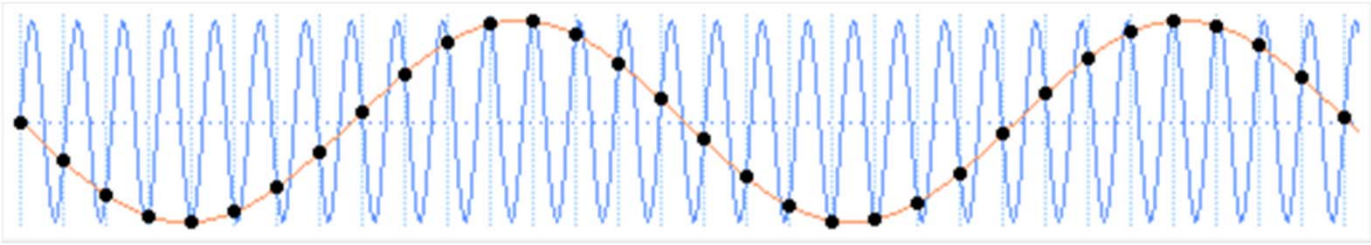


Nyquist Revisited

- Sampling at the correct rate (*2f*) yields actual signal
  - Always assume lowest-frequency wave that fits samples

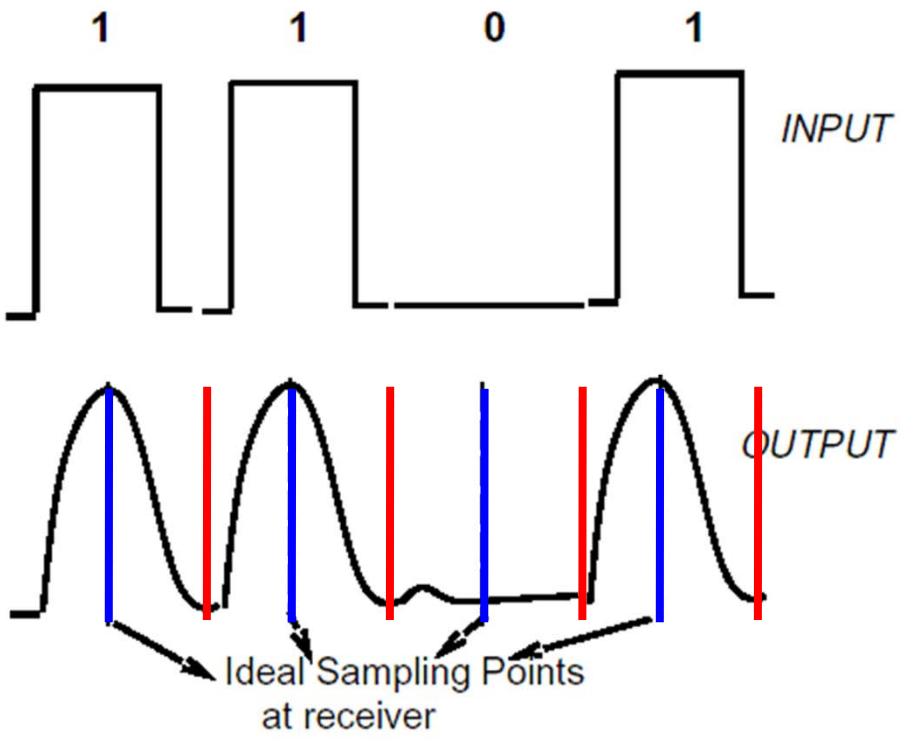


- Sampling too slowly yields aliases



The Importance of Phase

- Need to determine when to START sampling, too

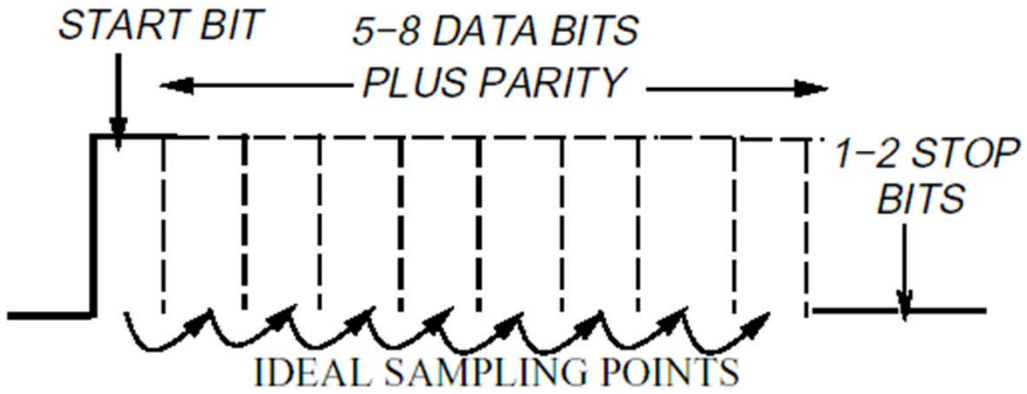


Clock Recovery

- Using a training sequence to get receiver lined up
  - Send a few, known initial training bits
  - Adds inefficiency: only *m* data bits out of *n* transmitted
- Need to combat clock drift as signal proceeds
  - Use transitions to keep clocks synched up
- Question is, how often do we do this?
  - Quick and dirty every time: asynchronous coding
  - Spend a lot of effort to get it right, but amortize over lots of data: synchronous coding

Asynchronous Coding

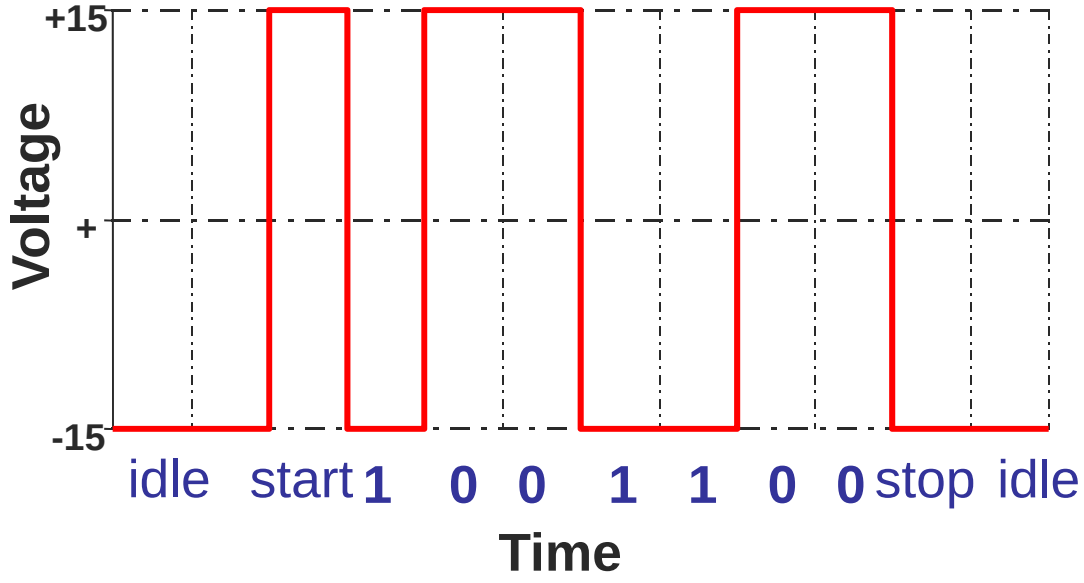
- Encode several bits (e.g. 7) together with a leading “start bit” and trailing “stop bit”
- Data can be sent at any time



- Start bit transition kicks of sampling intervals
  - Can only run for a short while before drifting

Example: RS232 serial lines

- Uses two voltage levels (+15V, -15V), to encode single bit binary symbols
- Needs long idle time – limited transmit rate

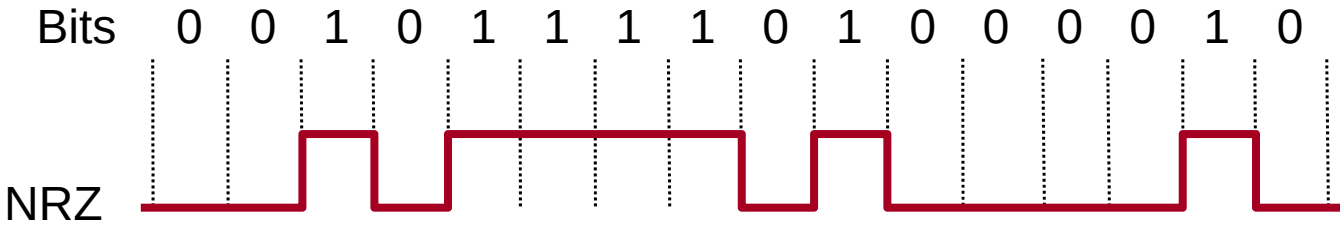


Synchronous Coding

- Asynchronous receiver phase locks each symbol
  - Takes time, limiting transmission rates
- So, start symbols need to be extra slow
  - Need to fire up the clock, which takes time
- Instead, let's do this training once, then just keep sync
  - Need to continually adjust clock as signal arrives
  - Phase Lock Loops (PLLs) ?
- Basic idea is to use transitions to lock in

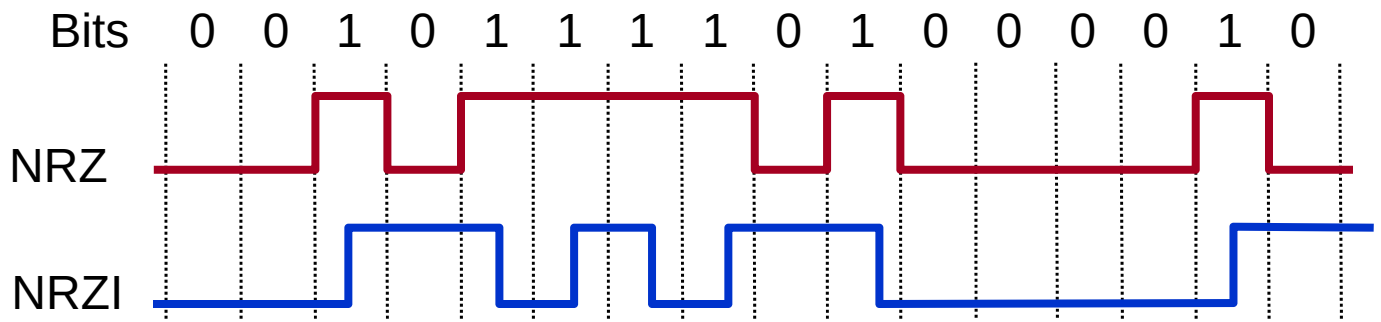
Non-Return to Zero (NRZ)

- Signal to Data
  - High ⇨ 1
  - Low ⇨ 0
- Comments
  - Transitions maintain clock synchronization
  - Long strings of 0s confused with no signal
  - Long strings of 1s causes *baseline wander*
    - » We use average signal level to infer high vs low
  - Both inhibit clock recovery



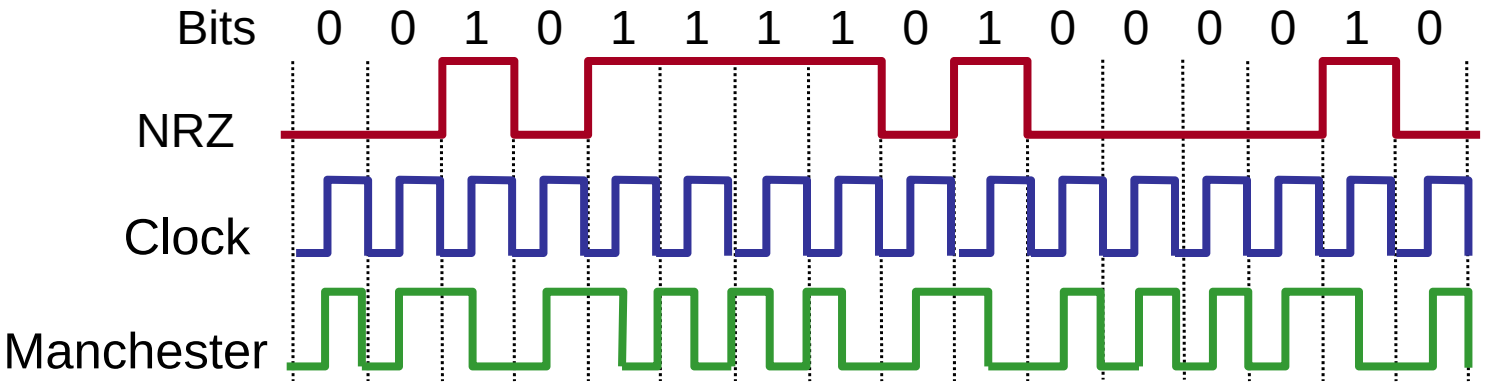
Non-Return to Zero Inverted (NRZI)

- Signal to Data
  - Transition ⇨ 1
  - Maintain ⇨ 0
- Comments
  - Solves series of 1s, but not 0s



Manchester Encoding

- Signal to Data
  - XOR NRZ data with senders clock signal
  - High to low transition ⇨ 1
  - Low to high transition ⇨ 0
- Comments
  - Solves clock recovery problem
  - Only 50% efficient (½ bit per transition)
  - Still need preamble (typically 0101010101… trailing 11 in Ethernet)



4B/5B (100Mbps Ethernet)

- Goal: address inefficiency of Manchester encoding, while avoiding long periods of low signals
- Solution:
  - Use five bits to encode every sequence of four bits
  - No 5 bit code has more than one leading 0 and two trailing 0's
  - Use NRZI to encode the 5 bit codes
  - Efficiency is 80%

4-bit	5-bit
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111

4-bit	5-bit
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101

Summary

- Two basic tasks: send and receive
  - The trouble is the channel distorts the signal
- Transmission modulates some physical carrier
  - Lots of different ways to do it, various efficiencies
- Receiver needs to recover clock to correctly decode
  - All real clocks drift, so needs to continually adjust
  - The encoding scheme can help a lot

For Next Class

- Framing and error detection
- Read 2.3-2.4
- HW #1 is assigned… due next Tues